

Gerenciamento de Configuração e Integração Contínua I

Semana 4

CSDV-246 -DevOps



Conteúdo

- Gerenciamento de configuração (CM)
 - Artefatos
 - Definição
 - Idempotência
 - Configuration Drift
- Integração contínua (CI) I
 - Introdução ao CI
 - Elementos principais
 - Sistema básico de IC
 - Práticas Essenciais
- Pipeline como código



Gerenciamento de configuração (CM)

Artefatos



- Um artefato é qualquer arquivo, componente ou elemento gerado ou usado durante o ciclo de vida de desenvolvimento de um projeto.
- Os seguintes elementos podem ser considerados artefatos:
 - **Código-fonte:** os arquivos de código que fazem parte do projeto.
 - **Arquivos binários:** programas ou aplicativos compilados.
 - **Dependências:** bibliotecas ou módulos externos que o projeto usa.
 - **Documentação:** Manuais, diagramas ou guias relacionados ao projeto.

Artefatos



- **Scripts de configuração:** Arquivos que automatizam tarefas de configuração ou implantação.
- **Modelos de infraestrutura:** Arquivos que definem como a infraestrutura deve ser configurada (exemplo: arquivos de estado).
- **Recursos multimídia:** Imagens, vídeos ou arquivos que fazem parte do projeto.

Definição de CM



- É uma das práticas de DevOps, através do qual garante que todos os artefatos do projeto sejam gerenciados de forma organizada, identificando-os de forma única, rastreando suas versões e controlando como eles são modificados e usados ao longo do desenvolvimento e implantação do projeto.
- Além disso, esta Prática **inclui o gerenciamento de ambientes ou ambientes** (desenvolvimento, teste, produção) para garantir que os sistemas operem de forma consistente e eficiente.
- Sem o CM, é praticamente impossível executar a integração contínua, o gerenciamento de versões e os fluxos automatizados de implantação para produção.

Definição de CM



- Esta prática não é apenas uma questão de escolher e implementar uma ferramenta como tal (Ansible, por exemplo).
- Não há dúvida de que a ferramenta é muito importante para esta Prática, mas o CM vai além de uma ferramenta específica.
- O que é realmente crucial, essencial, para o CM é APLICAR BOAS PRÁTICAS diariamente.

Definição de CM



- Se o processo de CM for sólido no ciclo de vida de desenvolvimento de software, as seguintes perguntas devem ser respondidas com um "SIM":
 1. Você poderia recriar completamente o ambiente de produção, excluindo dados de produção, do zero a partir dos ativos versionados em um VCS?
 2. Você poderia reverter seu aplicativo para um estado anterior e estável em caso de falhas?
 3. Você pode ter certeza de que todos os ambientes implantados em Produção, Preparo e Teste estão configurados exatamente da mesma maneira?

Definição de CM



- Se a resposta for "NÃO", então a organização está em risco. Nesse caso, é recomendável ter uma estratégia para armazenar linhas de base e controlar as alterações em:
 1. O código-fonte do aplicativo, scripts de construção, testes, documentação, scripts de banco de dados, bibliotecas e arquivos de configuração.
 2. Todos os ambientes de desenvolvimento, teste e produção
 3. A configuração associada a cada aplicativo em cada ambiente em que ele é executado, durante todo o ciclo de vida do aplicativo (compilação, implantação, teste, operação)

Definição de CM



- O CM cobre os seguintes pontos:
 1. Gerenciamento do processo de criação, implantação, teste e liberação do aplicativo.
 - Uso do controle de versão
 - Gerenciamento de dependências
 2. Gerenciamento das configurações de software
 3. Administração dos Ambientes

Controle de versão



- É importante armazenar todas as informações necessárias para recriar os ambientes de desenvolvimento, teste e produção nos quais o aplicativo é executado.
- Isso deve incluir informações de configuração para a pilha de software do aplicativo e os sistemas operacionais que compõem o ambiente, arquivos de zona DNS, configurações de firewall e assim por diante.
- No mínimo, você precisa de tudo o que precisa para recriar os binários do aplicativo e os ambientes em que eles são executados.

Controle de versão



- O objetivo é ter tudo o que pode mudar a qualquer momento durante a vida do projeto armazenado de forma controlada.
- Isso permite que você recupere um instantâneo exato do estado de todo o sistema, desde o ambiente de desenvolvimento até o ambiente de produção, em qualquer ponto do histórico do projeto.
- É útil até mesmo manter os arquivos de configuração dos ambientes de desenvolvimento da equipe de desenvolvimento no controle de versão, pois facilita o uso das mesmas configurações por todos os membros da equipe.

Controle de versão



- Além de armazenar o código-fonte e as informações de configuração, muitos projetos.
- Armazene também imagens binárias de seus servidores de aplicativos, compiladores, máquinas virtuais e outras partes de sua cadeia de ferramentas no controle de versão.
- Isso é fantasticamente útil, pois acelera a criação de novos ambientes e, mais importante, garante que as configurações básicas sejam totalmente definidas e, portanto, sabe que são boas e não irão falhar.

Controle de versão



- Algo que não é recomendado manter no controle de versão são arquivos binários.
- Em primeiro lugar, eles são grandes e criamos novos binários toda vez que eles passam nos testes.
- Em segundo lugar, novos binários podem ser gerados a partir do código-fonte, e o código-fonte geralmente muda constantemente à medida que o projeto avança, portanto, novos binários serão gerados para incluir o novo código.
- Depois que um artefato (por exemplo, uma versão de software) é criado, ele não deve ser modificado. Isso garante que a mesma versão do software seja implantada de forma previsível em diferentes ambientes.

Gerenciamento de dependências



- As dependências externas mais comuns no aplicativo são bibliotecas de terceiros.
- Além disso, eles podem ser considerados como dependências dos componentes ou módulos que nosso aplicativo precisa e que são desenvolvidos por outras equipes de desenvolvimento dentro da mesma empresa.
- É muito comum que as bibliotecas sejam implantadas na forma de arquivos binários e estes geralmente são atualizados com muita frequência

Gerenciamento de dependências



- **Bibliotecas externas:** Essas bibliotecas geralmente são instaladas globalmente em nosso sistema por meio de um gerenciador de pacotes, por exemplo:
 - APT
 - PIP
 - Composer
 - Maven
 - etc
- Baixar as dependências da Internet toda vez que compilamos e construímos nosso aplicativo **não é eficiente**. É melhor armazenar versões diferentes em algum lugar que seja acessível através da rede.

Gerenciamento de dependências



- Existem diferentes soluções para armazenar e controlar a versão de dependências na forma de binários:
 - Git LFS
 - JFrog Artifactory
 - Nexus OSS
 - GitHub Packages
 - GitLab Package Registry
 - Outros

Gestão de Ambientes



- Cada aplicativo que construímos depende de hardware, outro software, infraestrutura e outros sistemas para funcionar corretamente.
- A configuração do ambiente é tão importante quanto a configuração do aplicativo.
- Por exemplo:
 - Nosso aplicativo depende de um barramento de mensagens, esse barramento precisa ser configurado corretamente, caso contrário, nosso aplicativo não funcionará.

Gestão de Ambientes



- O objetivo é garantir que os ambientes de desenvolvimento, teste e produção sejam consistentes entre si.
- Os problemas geralmente surgem quando há diferenças entre os ambientes, fazendo com que algo funcione em um ambiente, mas falhe em outro.
- Esse problema de diferenças nas configurações dos ambientes é conhecido como **Configuration Drift**.



Configuration Drift

- Este termo descreve a situação em que a configuração dos ambientes (Desenvolvimento, Teste, Produção) está fora de sincronia ou há uma "incompatibilidade" da configuração desejada.
- Isso ocorre quando **alterações manuais** não documentadas ou **alterações** não **controladas** são feitas nos sistemas, levando a diferenças entre os ambientes.

Gestão de Ambientes



Causas comuns de Configuration Drift

- Alterações manuais ou não documentadas.
- Falta de automação.
- Atualizações ou patches fora de sincronia

Gestão de Ambientes



Soluções para Configuration Drift

- O desvio de configuração é um desafio significativo, mas pode ser significativamente minimizado com práticas de automação e DevOps, como gerenciamento de configuração e infraestrutura como código.
- **Controle de versão:** Certifique-se de que todas as alterações de configuração sejam versionadas e aplicadas de forma consistente em todos os ambientes.
- **Automação:** Automatize a configuração e o provisionamento de ambientes para garantir que todos os sistemas sejam configurados de forma idêntica.

Gestão de Ambientes



Automação da configuração do ambiente

- Para garantir que todos os ambientes sejam configurados da mesma maneira, é recomendável usar ferramentas de automação que apliquem as configurações de forma consistente em todos os ambientes.
- **Ferramentas de automação:** **Ansible**, Chef, Puppet, Scripting.
- Os scripts de configuração devem ser totalmente **repetíveis**, o que significa que podem ser executados em qualquer ambiente sem causar problemas. Isso evita erros resultantes de configurações manuais inconsistentes.

Gestão de Ambientes



Idempotência

- É a capacidade de executar uma operação ou aplicar uma configuração várias vezes sem alterar o resultado além da primeira aplicação.
- Em outras palavras, não importa quantas vezes a mesma configuração seja aplicada, o sistema sempre terminará no mesmo estado desejado, sem alterar o estado desejado.
- **Exemplo:** Se uma configuração especificar que um serviço deve estar em execução, uma operação idempotente primeiro verificará a integridade do serviço. Se o serviço já estiver ativo, ele não fará nenhuma alteração. Se não estiver ativo, ele irá iniciá-lo. Quando você executa o processo várias vezes, o estado final do sistema será o mesmo: o serviço em execução.

Gestão de Ambientes



Ansible

- Ele usa uma abordagem declarativa, o que significa que descreve o estado desejado do sistema em vez de como chegar a esse estado.
- As tarefas no Ansible são idempotentes, o que significa que você pode executá-las várias vezes sem se preocupar com a configuração incorreta do sistema ou com a duplicação do trabalho.
- Além disso, o Ansible possui uma ampla variedade de módulos que permitem executar diferentes tipos de tarefas, como instalação de pacotes, gerenciamento de serviços, configuração de usuários ou gerenciamento de arquivos.

Gestão de Ambientes



Casos de Uso do Ansible

- Um caso típico no Gerenciamento de Configuração com Ansible é automatizar a configuração de um servidor. Por exemplo, você pode usar um manual do Ansible para:
 - Instalar os pacotes necessários(NGINX, MySQL, etc.).
 - Configurar arquivos do sistema.
 - Configurar usuários e permissões.
 - Certifique-se de que os serviços necessários estejam em execução (como um servidor web ou banco de dados).

Integração Contínua I

Introdução ao CI



- O CI é uma prática empregada no Desenvolvimento de Software.
- Consiste em integrar código com frequência, mesclando pequenas (mas estáveis) alterações em uma ramificação de integração no VCS.
- O objetivo é detectar erros, bugs, falhas o mais cedo possível durante a integração do código.
- Um processo de Build, Build e Test é executado praticamente para verificar a estabilidade do código.

Introdução ao CI



- A integração de código é uma prática antiga no desenvolvimento de software.
- Usando metodologias tradicionais de desenvolvimento de software, as integrações de código costumavam ser executadas depois que as equipes de desenvolvimento implementavam suas funcionalidades por meses.
- As integrações costumavam ser extensas e tediosas, pois ao integrar depois de meses, existem centenas ou milhares de bugs, erros, detectados, e cada um deve ser corrigido para estabilizar o código.

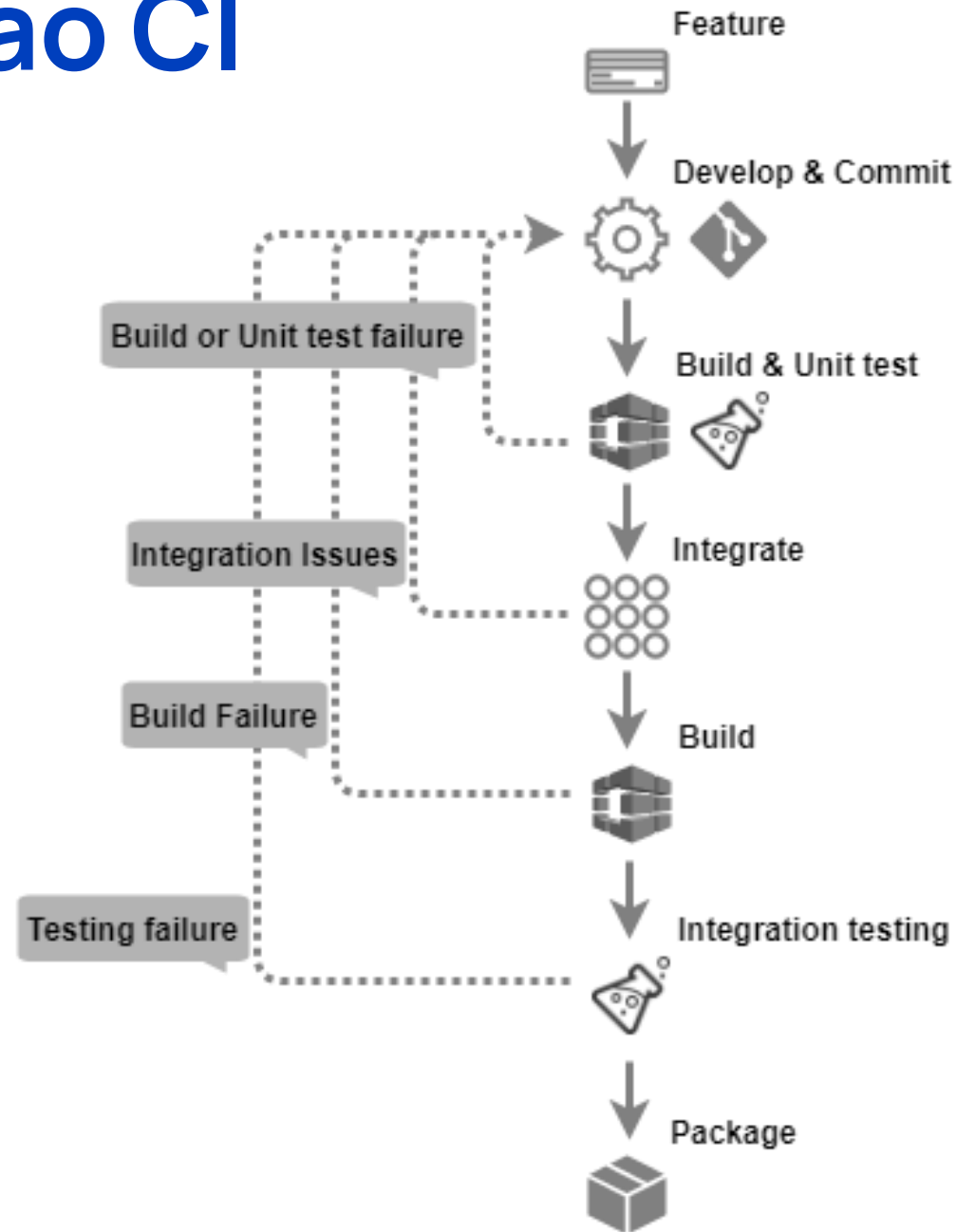
Introdução ao CI



- Atualmente trabalhamos com metodologias ágeis.
- Ao integrar pequenas alterações de código, é muito mais fácil e rápido detectar erros ou bugs e o tempo de integração é consideravelmente reduzido.
- Este processo pode ser executado manual ou automaticamente.
- Em um processo manual, um desenvolvedor é obrigado a executar o Build e o Teste do código integrado em um servidor de integração.

Introdução ao CI

Fluxo CI



Introdução ao CI



- O resultado final do fluxo CI é um "Package".
- Este package ou artefato contém o código incorporado depois de ter passado com êxito no teste de unidade.
- O fluxo pode ser automatizado usando uma ferramenta de Integração Contínua.
- Neste caso, são as chamadas "Pipelines".

Introdução ao CI



- Existem equipes de desenvolvimento que pensam que por ter uma ferramenta de Integração Contínua, já estão fazendo Integração Contínua, o que é um equívoco.
- **CI não é uma ferramenta**, é uma PRÁTICA que exige comprometimento e disciplina da equipe de desenvolvimento.
- É uma alta prioridade corrigir quaisquer alterações que tenham quebrado o aplicativo após a integração.
- Se cada desenvolvedor adotar essa disciplina, a melhoria na qualidade do aplicativo será perceptível.

Elementos de CI



- Existem diferentes elementos que ajudam a obter um fluxo de CI robusto e confiável.
 - Servidor de CI.
 - Estrategia de Branching
 - VCS
 - Build automatizado
 - Teste de unidade
 - Cobertura de código
 - Análise de código estático
 - Teste de integração
 - Repositório binário

Elementos de CI



Servidor de CI

- É uma ferramenta que automatiza o processo de integração contínua no desenvolvimento de software.
- O principal objetivo dessas ferramentas é ajudar as equipes a integrar o código de vários desenvolvedores com frequência e eficiência, garantindo que quaisquer alterações ou novos recursos possam ser testados e validados automaticamente.

Elementos de CI

Servidor de CI

- Alguns exemplos:
 - Jenkins
 - GitLab CI/CD
 - GitHub Actions
 - Circle CI
 - Azure Pipelines
 - Travis CI
 - Bamboo



Jenkins



GitLab CI



Bamboo



**Azure
DevOps**



GitHub Actions



Travis CI



circleci

Elementos de CI



Estrategia de Branching

- Revendo este ponto que vimos no Semana 1.
- Basicamente, eles são um conjunto de regras, etapas, que os desenvolvedores podem seguir para determinar como vão interagir com um repositório de código compartilhado.
- Exemplos:
 - GitHub Flow
 - GitLab Flow
 - Git Flow

Elementos de CI



VCS

- O Sistema de Controle de Versão é uma ferramenta que permite gerenciar e rastrear alterações no código-fonte ou em qualquer arquivo de um projeto.
- Mantém o histórico de versões e facilita a colaboração entre vários desenvolvedores.
- Exemplos:
 - Git
 - Subversion



Elementos de CI



Build Automatizado

- É um processo de desenvolvimento de software em que o código-fonte é automaticamente convertido em um produto executável ou distribuível sem intervenção manual.
- Ele inclui várias etapas, como compilar o código, executar testes e criar artefatos prontos para implantação.
- Exemplos:
 - Maven o Gradle
 - DotNet Build



Teste de unidade

- Esses testes se concentram na avaliação de partes do código de um programa, conhecidas como unidades. Uma unidade pode ser uma função, método, classe ou módulo, dependendo da linguagem de programação.
- O objetivo do teste de unidade é garantir que cada uma dessas unidades funcione corretamente.
- Exemplos de acordo com a linguagem de programação:
 - Jest para NodeJS.
 - Tox o unittest para Python.

Elementos de CI



Cobertura

- É uma métrica que mede a porcentagem de código que é executada e testada por testes unitários.
- Indica quais partes do código foram testadas para garantir que estejam funcionando corretamente. Também indica a porcentagem de código que não foi coberta por testes de unidade.
- Normalmente, as mesmas ferramentas de teste de unidade permitem que você obtenha o relatório de cobertura:
 - Jest para NodeJS.
 - Tox o unittest para Python.

Elementos de CI



Análise de Código Estático

- Um processo no qual o código-fonte de um aplicativo é examinado sem executá-lo.
- O objetivo é detectar erros, vulnerabilidades de segurança, violações de boas práticas ou possíveis melhorias no desempenho e na qualidade.
- Ele é executado antes que o código seja compilado ou executado.
- **Ferramentas:** SonarQube, Pylint, ESLint.

Elementos de CI



Teste de integração

- Eles são um tipo de teste de software usado para verificar se os diferentes módulos ou componentes de um aplicativo funcionam corretamente juntos.
- O objetivo principal é identificar defeitos na interação entre os componentes que estão integrados.
- Esses testes são realizados após a conclusão dos testes de unidade.
- **Ferramentas:** JUnit com Mock frameworks como Mockito, pytest o unittest, Postman, Selenium.

Elementos de CI



Repositório binário

- Eles são sistemas de gerenciamento de artefatos projetados para armazenar, controlar a versão e distribuir artefatos binários gerados no processo de desenvolvimento de software.
- Ao contrário dos sistemas de controle de versão, como o Git, que armazenam o código-fonte, os repositórios binários gerenciam artefatos derivados do código que precisam ser reutilizados ou distribuídos no ciclo de vida de desenvolvimento.
- **Ferramentas:** JFrog Artifactory, GitHub Packages, GitLab Packages, Nexus Repository, Git LFS (caso especial).

Sistema Básico de CI



- Deve-se notar que para fazer CI, **NENHUM SERVIDOR OU SOFTWARE DE CI NECESSÁRIO**. Como já vimos, o CI é uma prática, não uma ferramenta.
- Atualmente, temos muitas ferramentas à nossa disposição para automatizar nossos processos de integração, e elas são muito fáceis de usar.
- Essas ferramentas de CI nos permitirão automatizar nosso fluxo de CI nas chamados "Pipelines".
- Além disso, eles nos fornecem visualizações de logs de processos, envio de notificações de sucesso ou falha, relatórios de teste, etc.

Sistema Básico de CI



Pipeline

- No contexto do DevOps, um pipeline é uma sequência automatizada de etapas que incluem compilação, empacotamento, teste, integração e implantação de software, desde o código-fonte até a produção.
- Essa sequência automatizada é executada de forma recorrente, por exemplo, sempre que um novo código é integrado ao branch principal ou quando um Pull Request é criado.
- A execução de um pipeline é chamada de "build" e cada build tem um ID numérico sequencial atribuído pelo software de CI.

Sistema Básico de CI



O processo a ser seguido

1. Verifique se já existe uma compilação em execução. Em caso afirmativo, espere até que termine. Se falhar, você precisará trabalhar com o restante da equipe para corrigi-lo antes de integrar seu código.
2. Depois que a compilação for concluída com êxito, atualiza o código em seu ambiente local.
3. Compile seu código com as alterações mais recentes e verifique se os testes de unidade foram concluídos com êxito em seu ambiente local

Sistema Básico de CI



O processo a ser seguido

4. Se a compilação e os testes de unidade forem bem-sucedidos, crie uma solicitação de pull para integrar seu código ao branch principal.
5. Depois que o Pull Request for aprovado, uma nova compilação do pipeline de CI será executada automaticamente, incluindo seu código.
6. Se o pipeline falhar, você deverá corrigir o problema imediatamente em seu ambiente local e continuar esse processo da etapa 3, até que a compilação do pipeline seja bem-sucedida.

Práticas Essenciais



O objetivo do nosso sistema de CI é garantir que nosso software funcione, essencialmente, o tempo todo. Para garantir que esse seja o caso, aqui estão as práticas que devemos aplicar em nossas equipes:

- Não integre o código em cima de uma compilação quebrada ou com falha.
- Sempre execute todos os testes em seu ambiente local antes de integrar seu código.
- Espere que todos os testes terminem com sucesso antes de seguir em frente.

Práticas Essenciais



- Nunca deixe uma compilação quebrada, você deve corrigi-la para não bloquear o fluxo.
- Não desabilite (ou comente em linhas de) testes com falha.
- Assuma a responsabilidade por todas as falhas resultantes de suas alterações feitas no código.
- Meça a qualidade e a cobertura do código. Você não melhora o que não mede.
- Test-Driven Development.

Pipeline Como Código

Pipeline como Código



Pipeline:

- Vamos lembrar que um pipeline é uma sequência automatizada de etapas, cada etapa representa um processo manual de Desenvolvimento de Software.
- Existem ferramentas de CI que nos permitem criar pipelines através da interface gráfica, sem a necessidade de escrever código.
- Por exemplo: Jenkins e Azure Pipelines.



Pipeline como Código



- Pipeline as Code é uma prática de Desenvolvimento de Software, que consiste em descrever as configurações das etapas de um Pipeline de Integração Contínua e Entrega Contínua em arquivos de código.
- Linguagens de script com sintaxe YAML, JSON ou DSL (Domain-Specific Language) são geralmente usadas.
- Esses arquivos devem ter controle de versão no mesmo repositório que o código-fonte do aplicativo.
- Dessa forma, cada modificação no pipeline pode ser revisada e atualizada de forma colaborativa.

Pipeline como Código



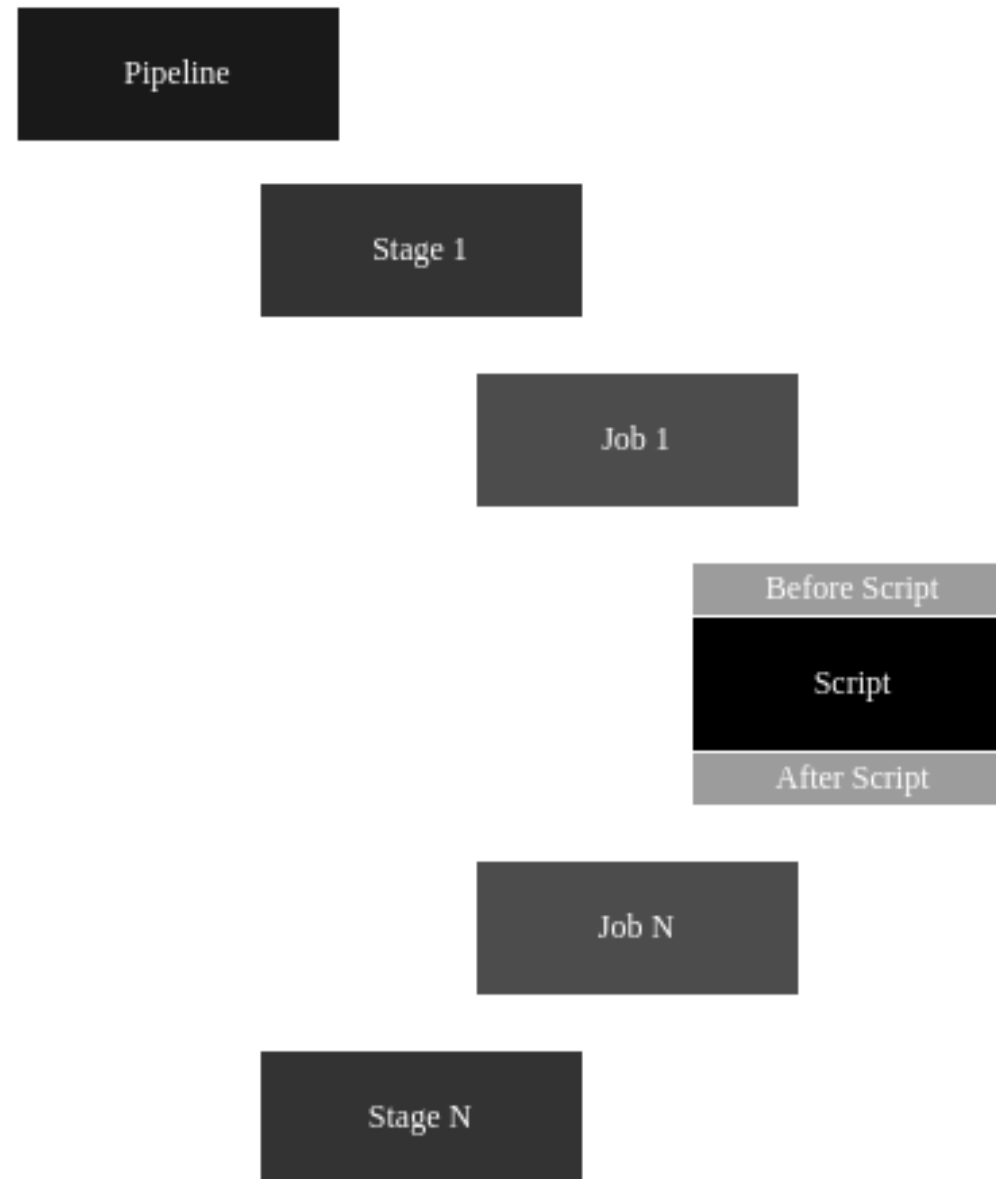
- **Versionável:** Cada alteração no pipeline pode ser versionada.
- **Automatizável:** Podemos automatizar todas as etapas do Desenvolvimento de Software.
- **Reprodutíveis:** Podemos replicar o pipeline em diferentes ambientes.
- **Colaborativo:** Os pipelines de CI podem ser modificados e aprimorados por qualquer membro da equipe de desenvolvimento.
- **Escalonável:** Podemos criar pipelines personalizados para diferentes projetos ou serviços, ajustando-se a necessidades específicas.

Pipeline como Código



Estrutura básica de uma Pipeline no GitLab

- O código deve ser escrito em um arquivo YAML
- O nome padrão é **`.gitlab-ci.yml`**





Pipeline como Código

Stage

- Um **estágio (Stage)** representa um estágio no pipeline.
- Cada estágio agrupa trabalhos (Jobs).
- Os estágios são executados em ordem sequencial, ou seja, um estágio deve ser concluído antes de passar para o próximo.
- Isso indica o fluxo do processo na pipeline.

stages:

- build
- test
- deploy



Pipeline como Código

Job

- Um trabalho (**job**) é uma tarefa específica que é executada dentro de um estágio.
- Cada job define quais ações serão executadas nessa fase específica, como compilar código, executar testes ou implantar um aplicativo.
- Os jobs podem ser executados em paralelo dentro do mesmo estágio.

```
build_job:  
  stage: build  
  script:  
    - echo "Compiling"  
    - make build
```

Pipeline como Código



Script

- O **script** é o script ou instruções que são executadas no contexto de um trabalho.
- Qualquer comando que deve ser executado é escrito para realizar a tarefa de trabalho.
- Eles geralmente são escritos em linguagem **bash** ou **shell** e serão executados em um runner.

script:

- echo "Compiling"
- make build

Pipeline como Código



```
stages:                                .gitlab-ci.yml
```

- build
- test
- deploy

```
build_job:
```

```
  stage: build
```

```
  script:
```

- echo "Compiling code"
- make build

```
test_job:
```

```
  stage: test
```

```
  script:
```

- echo "Running test"
- make test

```
deploy_job:
```

```
  stage: deploy
```

```
  script:
```

- echo "Deploying to Prod"
- make deploy

Pipeline como Código



Runners

- Os **runners** são componentes responsáveis pela execução de scripts de trabalho.
- Os runners podem ser instalados em ambientes de nuvem, máquinas virtuais, contêineres ou até mesmo em servidores locais.
- Eles suportam diferentes ambientes de tempo de execução, como Docker, shell, Kubernetes, entre outros.

Pipeline como Código



Tipos de Runners

- **GitLab-hosted runners (Shared runners):** Eles são runners gerenciados pelo GitLab e estão totalmente integrados ao GitLab.com. Por padrão, esses runners são habilitados para todos os projetos. Você pode desativar os runners se tiver a função de Proprietário para o projeto.
- **Self-managed runners (Specific Runners):** Você pode instalar os GitLab Runners e registrá-los no GitLab.com ou em sua própria instância. Para usar executores específicos, instale o GitLab Runner na infraestrutura que você possui ou gerencia.

Q&A